

Priora - Dokumentacija Koda

Verzija: 1.0.0 | Stack: React 19 + TypeScript · FastAPI + Python 3.12 · XGBoost (ML)

1. Backend

Backend je Python FastAPI aplikacija koja prima podatke o pacijentu, pokreće ih kroz trenirani ML model i vraća strukturiranu trijažnu procjenu. Ulazna tačka je backend/app/main.py.

app/main.py | app/routes/health.py

main.py inicijalizira aplikaciju, konfigurira CORS i registrira dva rutera pod prefiksom /api. health.py izlaže GET /api/health → {"status": "ok"} za provjeru dostupnosti servera.

app/routes/triage.py

Definiše POST /api/triage. TriageRequest je Pydantic model s Field validacijom za sve numeričke vitale. TriageResponse vraća: esi_level (1-5), label, hex color, confidence, probabilities i recommendation. Handler delegira na predict_triage().

Polje	Tip	Ograničenja
age / pulse / sbp / dbp	float	0-120 / 0-300 bpm / 0-300 / 0-200 mmHg
temperature / spo2 / resprate	float	30-45 °C / 0-100 % / 0-60 /min
pain / chiefcomplaint	float / str	0-10 / min. dužina 1

ml/preprocessing.py

Definira shemu karakteristika i gradi sklearn ColumnTransformer: numerički pod-pipeline (SimpleImputer medijan → StandardScaler) i kategorički (SimpleImputer → OneHotEncoder s handle_unknown="ignore", čime se nepoznate žalbe tretiraju kao red nula).

ml/train_real.py

Skripta za treniranje (python -m ml.train_real). extract_chief_complaint(df) skenira cc_* kolone i pretvara aktivnu zastavicu u string žalbe. load_and_prepare() čita 5v_cleandf.rdata, konvertuje temperaturu F→C i filtrira na ESI 1-5. train() dijeli podatke 80/20, gradi pipeline(preprocesor, XGBClassifier) i snima ga u triage_model.joblib.

ml/predict.py

_load_model() učitava i kešira triage_model.joblib. predict_triage(TriageInput) kreira jednoredni DataFrame, poziva predict() (klasa 0-bazirana, +1 → 1-5), predict_proba() (zaokruženo na jednu decimalu), traži preporuku u RECOMMENDATIONS i vraća TriageResult dataclass.

2. Frontend

React 19 + TypeScript SPA izgrađena s Vite-om. Komunicira s backendom jednim POST pozivom. Layout koristi Tailwind CSS v4 i shadcn/ui..

src/models/ | src/constants/

models/triage.ts definira TriageRequest i TriageResponse kao čiste TypeScript interfejsse koji ogledaju Pydantic modele polje po polje. constants/vitals.ts eksportuje VITAL_STATUS_CFG (mapiranje statusa na Tailwind klase) i DEFAULT_FORM za resetovanje obrasca.

constants/esi.tsx eksportuje ESI_CONFIG: Record<number, EsiCfg> s vizualnom konfiguracijom za ESI 1-5. constants/chiefComplaints.ts sadrži 200 objekata {value, label} sortiranih abecedno.

src/lib/

vitals.ts eksportuje getVitalStatus(name, value): VitalStatus | null - switch po nazivu polja s kliničkim pragovima; null za nulte vrijednosti. print-report.ts eksportuje buildPrintHtml(patientName, form, result): string koji generira self-contained A4 HTML dokument s inline CSS-om, gridom vitalnih znakova, ESI kutijom i linijama za potpis; otvara se putem window.open().

src/api/triage.ts

submitTriage(data: TriageRequest): Promise<TriageResponse> šalje post<TriageResponse> na /api/triage.

src/components/TriageForm.tsx

Root komponenta, vlasnik stanja: form, result, patientName, isLoading, error. handleSubmit validira chiefcomplaint prije API poziva; handleReset vraća DEFAULT_FORM; handlePrint poziva buildPrintHtml.

src/components/triage/

VitalField.tsx prikazuje numerički input s live status bedžom; koristi type="text" s lokalnim stanjima focused i draft kako bi izbjegao problem vodećih nula. PainSlider.tsx renderira range input 0-10 s CSS linear-gradient track fillom i verbalnim labelom (Bez bola / Blaga / Umjerena / Značajna / Jaka / Najgora). ChiefComplaintSelect.tsx je searchable combobox koji razlikuje query string (prikazan dok je dropdown otvoren) i odabrani model string proslijeđen roditelju; onMouseDown sprječava prerano zatvaranje dropdowna. TriageResult.tsx prikazuje ESI bedž, preporuku i distribuciju vjerovatnoće kao progress barove bojane prema ESI_CONFIG. HelpPanel.tsx je sklopivi panel s referentnim opsezima za sva polja i legendom boja.

3. Tok podataka i horizontalni aspekti

End-to-end tok zahtjeva

handleSubmit() validira chiefcomplaint → submitTriage(form) šalje post zahtjev → FastAPI Pydantic validacija → predict_triage(): _load_model() (keširano), Pipeline.predict() + predict_proba(), TriageResult dataclass → TriageResponse JSON → setResult(res) → TriageResult komponenta. TypeScript interfejsi u src/models/ i Pydantic modeli u app/routes/triage.py moraju biti ručno usklađeni jer nema generisanja koda.

Pragovi vitalnih znakova

getVitalStatus() implementira dvostruku klasifikaciju isključivo za vizualni UI feedback - ne utiče na ML predikciju.

Polje	Opseg upozorenja	Kritični opseg
pulse	60-100 bpm	< 40 ili > 130 bpm
sbp / dbp	90-140 / 60-90 mmHg	< 80/>180 ili < 50/>110 mmHg

temperature	36,0-38,0 °C	< 35,0 ili > 39,5 °C
spo2 / resprate	≥ 95 % / 12-20 /min	< 90 % / < 8 ili > 30 /min
pain	5-7	≥ 8

Generisanje print izvještaja

buildPrintHtml() producira self-contained HTML s inline CSS-om: ljepljivi toolbar s dugmadima za print/zatvaranje, A4 kartica, grid vitalnih znakova, ESI kutija, probability barovi i linije za potpis. Prozor se zatvara putem window.onafterprint. Žalba se razrješava kroz CHIEF_COMPLAINTS.find() kako bi se prikazivao "Chest Pain" umjesto "chestpain".

Poznati nedostaci i prijedlozi unapređenja

Nedostatak	Uticaj	Prijedlog ispravke
pain nije u podacima za treniranje	Nema efekta na predikciju	Retrenirati s bolom ili sintetičkom augmentacijom
Ručno retreniranje	Model zastari	CI job ili zakazana skripta

Zaključak

Priora predstavlja funkcionalan prototip bolničkog sistema za trijažnu podršku koji uspješno integriše moderne web tehnologije s mašinskim učenjem. Trodijelna arhitektura - React frontend, FastAPI backend i XGBoost model - omogućava jasno razdvajanje odgovornosti i relativno jednostavno proširivanje sistema. Validacija na Pydantic i TypeScript nivou osigurava integritet podataka kroz cijeli stack, dok lazy loading i keširanje modela doprinose efikasnosti pri zaključivanju. Identificirani nedostaci, posebno izostanak varijable pain iz skupa za treniranje, prioritetna stavka za sljedeću iteraciju. Uvođenjem varijabli okruženja, error boundary-ja i automatiziranog ciklusa retreniranja, sistem bi bio spreman za testiranje u produkcionom okruženju.